

Table des matières

Corrigé 01 — Le métier, la chaîne et la RAW reçue	1
Questions courtes	1
Questions pratiques	3
Diagnostics	5
Questions à développer	6
Mises en situation	6

Corrigé 01 — Le métier, la chaîne et la RAW reçue

Comment lire ce corrigé. Il donne la réponse et le pourquoi, jamais une notion neuve : tout ce qu'il emploie a été enseigné dans la Théorie 01 ou pratiqué dans le Labo 01. Quand une réponse repose sur un geste du labo, le renvoi le nomme.

L'énoncé est dans Exercices_01.md.

Questions courtes

1. Les trois métiers. L'ingénieur de données demande comment amener la donnée à l'échelle, de façon fiable ; l'ingénieur analytique, comment transformer le brut en tables fiables et compréhensibles ; l'analyste, ce que la donnée raconte. Le partage ne tient pas à l'outil, puisque les trois écrivent du SQL, mais à la question.
2. Le geste signature. Appliquer à la transformation des données les pratiques du génie logiciel : versionnement dans Git, tests automatisés, documentation générée depuis le code, intégration continue. C'est la différence entre un SQL qui marche sur un poste et une plateforme qu'on tient en production, et c'est ce que le mot « ingénieur » désigne.
3. Les deux lettres qui bougent. Extraire, transformer, charger. Ce sont le T et le L qui échangent leur place : l'ETL transforme avant de charger, l'ELT charge d'abord et transforme dans l'entrepôt.
4. L'avantage décisif. Garder le brut. On le mesure le jour où une règle métier change, ou quand on découvre une erreur dans une transformation écrite six mois plus tôt : en ELT, on rejoue la transformation sur un brut qu'on a encore ; en ETL, la donnée d'origine n'a jamais été chargée et il faut retourner voir la source, parfois purgée depuis.
5. Les couches du socle. Le socle est RAW, STAGING et MARTS. INTERMEDIATE et PRESENTATION ne viennent qu'au besoin : la première quand un calcul métier doit servir à plusieurs endroits, la seconde quand un rapport précis réclame une forme particulière.

6. La responsabilité du STAGING. Il renomme, type, décode, et tire l'état courant de chaque source. Ce qu'il ne fait jamais : joindre deux sources. Un STAGING qui joint devient un endroit où l'on fait du métier, et le métier a déjà ses couches.
 7. Sauter une couche. Le décodage ou le typage de cette table vit alors à deux endroits, et le jour où il change, l'un des deux est oublié.
 8. Couche et schéma. Parce que « couche » désigne une responsabilité dans la chaîne, pas un objet de la base. Une plateforme qui reçoit dix sources peut avoir dix schémas de staging, un par source, et ils forment tous ensemble la couche STAGING.
 9. L'étage du milieu. L'argent d'un lakehouse joint volontiers plusieurs sources ; le STAGING s'y refuse. C'est l'écart qui compte entre les deux vocabulaires, et celui qui coûte cher quand une équipe les confond.
 10. Les quatre produits. La couche RAW (22 tables) ; les séries horaires de l'usine, dans le schéma argent ; le journal des chargements, `pipeline.chargements` ; et `LIVRAISON.md`, le document qui décrit tout cela. Le quatrième est celui qu'on oublie, et c'est le seul qui permette de distinguer une livraison complète d'une livraison tronquée.
 11. Le premier geste. Compter, et comparer ses comptes à ceux qui sont annoncés. Pas transformer.
 12. La porte d'entrée. Par le kit de départ de la formation, qui reconstruit exactement ce que Data Engineering aurait livré, en une commande. *Labo : `Labo_01.md`, étape 1.*
 13. Les trois colonnes. `_fichier`, le chemin du fichier chargé depuis le projet ; `_lot`, la livraison, mois pour un extrait, jour pour la télémétrie, `complet` pour un export entier ; `_charge_le`, l'heure du chargement.
 14. À quoi sert `_lot`. Devant deux lignes qui portent le même numéro de commande, elle seule dit laquelle est la plus récente. Ni le contenu des lignes ni leur ordre dans la table ne le disent.
1. Les trois défauts. Le même objet livré plusieurs fois ; des codes que personne n'a décodés ; des textes qui ne sont pas du texte, dates et montants en particulier.
 2. Le contrat, pas la dette. Parce qu'elle aurait détruit une information : quelle version d'une commande était connue à quelle date. Le labo l'a montré sur la commande 0010006931, qui valait 6 079,02 \$ dans l'extrait de février et 5 556,72 \$ dans celui de mars. Une RAW dédoublonnée n'aurait gardé qu'un de ces deux chiffres, et personne n'aurait pu retrouver l'autre. *Labo : `Labo_01.md`, étape 4.*
 3. Les comptes. 22 tables, 513 857 lignes. *Labo : `Labo_01.md`, étape 2.*
 4. Les types. `VARCHAR` pour les 250 colonnes qui viennent d'un fichier, sans exception ; `TIMESTAMP WITH TIME ZONE` pour les 22 colonnes `_charge_le`, une par table, écrites par le chargeur ; `JSON` pour deux colonnes, les réclamations et la télémétrie, livrées en JSON et gardées telles quelles. *Labo : `Labo_01.md`, étape 3.*
 5. La date vide. 00000000, une chaîne de huit zéros. Ce n'est pas `NULL` parce que le fichier livré porte ces huit caractères et que la RAW garde ce qui a été livré : mettre `NULL` à la place serait déjà une transformation.

6. Le maximum faux. Sur `netwr`, la valeur nette d'une commande. Le maximum en texte est 9 996,26 et le maximum réel 293 478,95, soit un facteur trente. La cause est l'ordre alphabétique : 9996.26 commence par un 9, et aucune chaîne ne le dépasse tant qu'on compare caractère par caractère. *Labo : Labo_01.md, étape 6.*
 7. Le dépôt. Les artefacts de dbt (`target/`, `dbt_packages/`, `logs/`), parce qu'ils se reconstruisent par une commande et n'ont rien à faire dans un historique ; et les fichiers de connexion (`profiles.yml`, `.env`), parce qu'ils porteront un mot de passe dès qu'une plateforme infonuagique sera visée. *Labo : Labo_01.md, étape 7.*
 8. Le terrain réel. La météo, tirée des archives publiques d'Environnement et Changement climatique Canada. Tout le reste sort d'un générateur, ce qui rend chaque chiffre des cours reproductible.
-

Questions pratiques

P1. Compter une table.

```
select count(*) as lignes from raw.garanties_visites;
```

Attendu : 53 865, ce qu'annonce `LIVRAISON.md`, section 2.2.

P2. L'inventaire.

```
select table_name, estimated_size as lignes_estimees
from duckdb_tables()
where schema_name = 'raw'
order by table_name;
```

Vingt-deux lignes. `estimated_size` est l'estimation que DuckDB tient pour son planificateur : elle suffit à un inventaire, pas à un contrôle. *Labo : Labo_01.md, étape 2.*

P3. Le total, autrement.

```
select
  (select count(*) from raw.erp_kna1)
+ (select count(*) from raw.erp_mara)
+ (select count(*) from raw.erp_vbak)
+ (select count(*) from raw.erp_vbap)
+ (select count(*) from raw.erp_zserie) as lignes_erp;
```

Attendu : 150 367. Un `union all` suivi d'une somme donne le même résultat ; ce qui compte est de passer par `count(*)`, qui lit la table, et non par l'estimation.

P4. Les doublons d'une autre table.

```
select
    count(*) as objets,
    count(distinct objet ->> 'reclamation_id') as reclamations
from raw.garanties_reclamations;
```

Attendu : 5 623 objets pour 3 348 réclamations. L'opérateur ->> tire une valeur d'une colonne JSON et la rend en texte. La cause des doublons est la même que pour les commandes, en plus systématique : le portail renvoie l'objet complet à chaque changement de statut.

P5. Le lot le plus récent.

```
select _lot, count(*) as lignes
from raw.erp_kna1
group by 1
order by _lot desc;
```

Quarante-neuf lots, un par extrait mensuel, plus 2023-03_bis. Les plus récents portent 156 lignes, soit le référentiel complet des concessionnaires à cette date. Le tri sur un texte AAAA-MM donne ici le bon ordre chronologique, parce que le format est à largeur fixe et que les mois portent leur zéro de tête.

P6. Typage d'une date.

```
try_strptime(nullif(aedat, '00000000'), '%Y%m%d')
```

Trois pièces, et chacune est nécessaire. nullif traite la date vide comme une absence. strptime donne le format, que DuckDB ne devine pas sur 20230302. Le préfixe try_ rend NULL plutôt que de faire tomber la requête si une valeur inattendue se présente. Vérification sur la table entière : 2 308 dates obtenues sur 26 538 lignes, les autres étant les commandes jamais modifiées.

P7. Le montant juste.

```
select
    count(*) as commandes,
    sum(netwr::decimal(18, 2)) as valeur_nette
from raw.erp_vbak
where _lot = '2023-02';
```

Attendu : 577 commandes, 14 290 720,89 \$. Le type decimal et non double : un montant se compte au cent près, et l'arithmétique en virgule flottante n'est pas exacte.

P8. Une ligne, son origine.

```
select vbeln, _fichier, _lot, erdat, aedat, netwr
from raw.erp_vbak
where vbeln = '0010006931'
order by _lot;
```

Trois lignes, aux lots 2023-02, 2023-03 et 2023-03_bis. *Labo : Labo_01.md, étape 4.*

Diagnostics

E1. Le compte qui ne tombe pas. `count(*)` compte les lignes, pas les commandes, et la RAW garde chaque livraison d'une même commande. L'annonce surestime de 1 055, soit 26 538 au lieu de 25 483.

```
select count(distinct vbeln) as commandes from raw.erp_vbak;
```

L'erreur est instructive parce qu'elle ne se voit pas : 26 538 est un nombre plausible, et rien ne signale qu'il est faux. C'est le genre d'écart qu'un test de la couche MARTS attrapera au Cours 05.

E2. Le tri qui ment. `netwr` est du texte, et `order by` sur du texte trie alphabétiquement. 9996.26 passe devant 293478.95 parce que 9 est après 2. Mesuré : le tri en texte rend 9 996,26 puis quatre fois 9 996,20 ; le tri sur le nombre rend 293 478,95 puis 286 928,91.

```
select vbeln, netwr::decimal(18, 2) as valeur_nette
from raw.erp_vbak
order by netwr::decimal(18, 2) desc
limit 10;
```

E3. La conversion silencieuse. La colonne n'est pas vide : 2 308 lignes portent une vraie date. `try_cast(aedat as date)` échoue sur toutes, y compris celles-là, parce que 20230302 n'est pas écrit dans un format que DuckDB reconnaît d'office. Le `try_` transforme l'échec en NULL, si bien que la requête rend 0 sans le moindre message.

```
select count(try_strptime(nullif(aedat, '00000000'), '%Y%m%d')) as dates_valides
from raw.erp_vbak;
```

E4. Le décodage inventé. Z001, Z002 et Z003 n'existent pas dans cette colonne : `kukla` porte 01, 02 et 03. Le `case` ne trouve donc aucune correspondance et rend NULL sur les 156 concessionnaires. Rien ne plante, parce qu'un `case` sans `else` rend NULL, et un test d'ensemble de valeurs ne voit pas les NULL : il les ignore. Pour le constater, il faut compter les valeurs obtenues, et non se fier au fait que le modèle tourne.

```
select palier, count(*) from le_modele group by 1;
```

La règle qui aurait évité l'erreur : les noms de colonnes et les codes sortent du `_univers/boreal/DICTIONNAIRE.md`, jamais de la mémoire ni d'un autre support. *Labo : Labo_01.md, étape 5.*

E5. Le verrou. Un fichier DuckDB ouvert par un processus bloque tous les autres, même en lecture seule. Ce qui le tient est très probablement sa propre ligne de commande DuckDB, restée ouverte dans un autre terminal, ou une session laissée dans un éditeur. La parade : la fermer par `.exit` avant de lancer un script qui écrit. C'est la règle du « un seul processus à la fois » du labo.

Questions à développer

A. La frontière. Ce qu'on perdrait est une information, pas du confort. Décoder à l'ingestion fige le sens d'un code au jour du chargement : si le palier 03 change de nom l'an prochain, les lignes déjà chargées porteraient l'ancien libellé et les nouvelles le nouveau, dans la même colonne, sans que rien ne le signale. Dédoublonner à l'ingestion détruit l'historique des versions, et avec lui toute capacité à répondre à « que savait-on au 28 février ? ». La frontière est donc placée là où elle est pour une raison de fond : la RAW est un enregistrement de ce qui est arrivé, et un enregistrement ne se corrige pas, il s'interprète. Ce que cela dit de la notion de couche : une couche n'est pas une étape de nettoyage de plus en plus poussée, c'est une responsabilité distincte. RAW répond de la fidélité, STAGING de la lisibilité, MARTS de la modélisation. Confondre les trois donne une plateforme où personne ne sait plus quelle transformation a eu lieu où.

B. Le document de livraison. Il protège contre la livraison silencieusement incomplète : un chargement interrompu, un fichier manquant, une source qui n'a pas répondu. Ce sont des pannes qui ne lèvent aucune erreur, parce que du point de vue du chargeur tout ce qui est arrivé a été chargé. Il ne protège pas contre une donnée fausse mais complète : si l'ERP a produit des montants erronés, les comptes tomberont juste et le document ne dira rien. Dans une équipe sans ce document, chacun découvre les caractéristiques de la livraison à sa manière et à son moment, en général au milieu d'un incident ; les mêmes questions se reposent tous les six mois, et la réponse dépend de qui est encore là pour s'en souvenir.

C. (avancé) Deux vocabulaires. L'équivalence est dangereuse parce qu'elle autorise sans le dire une chose que la couche STAGING interdit : la jointure. Concrètement, un développeur qui croit que son STAGING est « de l'argent » écrira une vue de staging des commandes qui joint déjà le concessionnaire pour en ramener le nom et la province, puisque c'est ce qu'on fait à l'argent. La vue devient commode, tout le monde s'en sert, et la jointure se retrouve en amont de toute la chaîne. Le jour où l'on veut l'état du concessionnaire à la date de la commande plutôt qu'aujourd'hui, il faut défaire cette jointure dans chaque modèle qui en dépend. Le coût n'apparaît donc pas au moment de la faute, mais des mois plus tard, quand la dimension à évolution lente arrive.

Mises en situation

M1. Coopérative laitière de Sainte-Perpétue. Chaque fichier reçu est gardé tel quel, avec son horodatage de réception et son nom de fichier : c'est la RAW, et les trois colonnes de lignage y sont indispensables, puisque le même fichier arrive parfois deux fois. Rien n'est corrigé à l'ingestion, pas même les volumes contestés : le fichier corrigé est une livraison de plus, et la version qui vaut se choisit au STAGING, en gardant la plus récente pour chaque couple ferme-jour. Les températures aberrantes ne se suppriment pas non plus, elles se marquent à l'étage suivant. Ce qu'on exige du transporteur : un identifiant de livraison, ou à défaut une convention de nommage qui distingue une correction d'un renvoi du même fichier. Sans cela, la règle « la plus récente gagne » n'est pas applicable, et c'est le point que le Cours 03 rendra concret.

M2. (avancé) Transport Baie-des-Sables. D'abord, établir la vérité de ce qui existe : compter les sources, retrouver d'où vient le chiffre de la page d'accueil, et écrire ce que chaque rapport calcule réellement. On ne peut pas reprendre ce qu'on n'a pas inventorié, et l'indicateur à trois valeurs est le symptôme le plus utile, parce qu'il montre où sont les définitions concurrentes. Ensuite, poser la couche MARTS avec une seule définition de l'indicateur, et y brancher un rapport, un seul, pour prouver que la chaîne tient. Ensuite seulement, migrer les autres rapports un par un. Ce qu'on laisse pour la fin : débrancher les accès directs aux tables brutes. C'est tentant de commencer par là, parce que c'est la faute la plus visible, mais couper l'accès avant d'avoir un remplacement qui donne les mêmes chiffres transforme un problème de qualité en interruption de service, et l'équipe perd la confiance dont la reprise a besoin.

M3. La livraison silencieuse. Le fait que rien n'ait échoué est l'indice principal : une baisse de 40 pour cent sans erreur oriente vers une livraison incomplète plutôt que vers une transformation cassée, puisqu'une transformation cassée lève en général quelque chose. La démarche va donc de l'amont vers l'aval. Compter les tables de la RAW et comparer au document de livraison, exactement comme au labo. Si les comptes sont bons, regarder `_lot` et `_charge_le` : une source dont le dernier lot date d'avant-hier est passée sous silence, et le chargeur n'a rien à signaler puisque rien n'est arrivé. Si la livraison est complète, alors seulement descendre dans les modèles, en comparant les comptes couche par couche pour trouver l'étage où les lignes disparaissent.